
Modular Programming – Practical Work

Week 7

Recap exercise about inheritance and composition,
introduction interface, abstract class

Notations: W1Q1 = Week 1, Questions of type 1

Question types:

- Type 1: theory, concepts *knowledge*
- Type 2: understanding typically based on code reading *understanding*
- Type 3: producing yourself code *know-how*

A moodle deposit space is open every week for you to upload your solutions. Solutions of the exercises are provided one week later.

Important points:

- 602_F and 602_D classes should upload solutions **before Monday 24h00 the following week**
- **2 or 3 practical work will be corrected** and will count for one quarter of the intermediate test note

W7Q1 – 1

1. Give 3 differences between an abstract class and an interface ?

.....

.....

.....

.....

.....

2. Under which conditions can we implement an interface without coding the methods defined by the interface ?

.....

.....

.....

.....

.....

3. How to impeach that a class is extended by a sub-class ?

.....

.....

.....

.....

.....

.....

4. When defining a member variable, what is the difference between

- `public static final`
- `public static`
- `public final`
- `private static final`

.....

.....

.....

W7Q2 – 1

Analyse the following code and identify the potential error(s). If you find error(s), indicate the solutions to correct them.

```
class Base {  
    private int a = 0;  
    abstract void addValueToA();  
    int getA(){  
        return a;  
    }  
}
```

```
class Derived extends Base {  
    void addValueToA(){  
        a += 20;  
    }  
}
```

W7Q3 – 1 Enum

Several fruits in an orchard are ready to be picked. There is a given number of n workers collecting m different fruits.

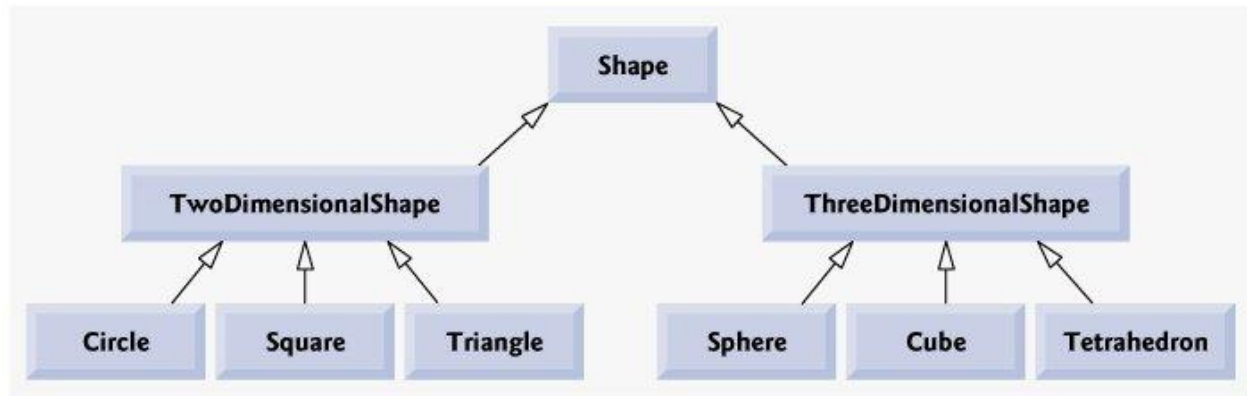
- Use an enumeration for the fruits. Implement the enum separately from all other classes.
- Use constants for the total number of workers and for the overall amount of fruits to be picked
- Generate a two-dimensional array corresponding to the given number of workers and the number of available fruits ($m \times n$ array). Use the array to store the collected amount of fruits for each worker and each fruit.
- Randomly chose a worker as well as a fruit. This process is supposed to take place p times (p is the overall amount of fruits to be picked - use the above specified constant). Increment the corresponding value in the available array.
- After the picking process has finished, print out the result: for each worker supply all fruits and the collected amount.
- Do not use the names of the fruits as Strings - use the implemented enum instead.
- Add some more fruits to the enumeration. No additional changes to the source code are supposed to be necessary to run the application with the enlarged number of fruits.

Here is the output that you should obtain :

```
worker 1 collected:
48 ORANGES
48 APPLES
49 STRAWBERRYS
55 BANANAS
worker 2 collected:
57 ORANGES
44 APPLES
50 STRAWBERRYS
47 BANANAS
worker 3 collected:
57 ORANGES
61 APPLES
30 STRAWBERRYS
49 BANANAS
worker 4 collected:
65 ORANGES
56 APPLES
44 STRAWBERRYS
42 BANANAS
worker 5 collected:
54 ORANGES
49 APPLES
51 STRAWBERRYS
44 BANANAS
```

W7Q3 – 2 Inheritance : abstract class

Implement a shape hierarchy such as the one shown below.



Each `TwoDimensionalShape` should contain method `getArea()` to calculate the area of the 2 dimensional shape. Each `ThreeDimensionalShape` should have methods `getArea()` and `getVolume()` to calculate the surface area and volume, respectively, of the 3 dimensional shape.

Create a program that uses an array of `Shape` references to objects of each concrete class in the hierarchy. The program should print a text description of the object to which each array element refers. Also, in the loop that processes all the shapes in the array, determine whether each shape is a `TwoDimensionalShape` or a `ThreeDimensionalShape`. If it is a `TwoDimensionalShape`, then display its area. If it is a `ThreeDimensionalShape`, display its area and volume.

Furthermore, you have the following constraints:

- A user of your classes should not be able to create an instance of `Shape` nor `TwoDimensionalShape` nor `ThreeDimensionalShape`.
- If a user wants to add a new subclass of `TwoDimensionalShape` or `ThreeDimensionalShape`, the implementation of `getArea()` is mandatory.
- If a user wants to add a new subclass of `ThreeDimensionalShape`, the implementation of `getVolume()` is mandatory.